LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek



NAMA: HUMAIRAH NURHAMIDAH

NIM: 103132400038

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman berorientasi objek (*Object Oriented Programming*/OOP) adalah paradigma pemrograman yang membangun sistem dari komponen-komponen berupa objek. Setiap komponen mengenkapsulasi data dan fungsi dalam satu kesatuan, dapat mewarisi atribut dan sifat dari komponen lain, dan saling berinteraksi. Pendekatan ini berbeda dari pemrograman prosedural yang berfokus pada urutan prosedur atau fungsi, bukan pada objek.

B. Guided

1. Abstraksi

```
package humaira.projectmodul03.guided.Abstraksi;

public class Abstraksi {

    public static void main(String[] args){

        Kucing aren = new Kucing();

        aren.nama = "Aren Gentong";

        aren.ras = "Mujaer";

        aren.bersuara();

    }

}
```

kucing

```
package humaira.projectmodul03.guided.Abstraksi;

public class Kucing {
    String nama;
    String ras;

    void bersuara(){
        System.out.println(nama + ras + "meow");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModl03 ---
Aren GentongMujaermeow
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Enkapsulasi

```
package humaira.projectmodul03.guided.enkapsulasi;

public class Enkapsulasi {
    public static void main(String[] args){
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}
```

Rekening

```
package humaira.projectmodul03.guided.enkapsulasi;

public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo(){
        System.out.println("Saldo Anda: " + saldo);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModl03 ---
Saldo berhasil ditambahkan
Saldo Anda: 20000000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Circle

```
package humaira.projectmodul03.guided.Circle01;

public class Circle01 {
    public static final double PI = 3.14159;

    public static double radiansToDegrees(double rads) {
        return rads * 180 / PI;
    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }

    public static void main(String[] args) {
        Circle01 lingkaran = new Circle01();
        lingkaran.r = 7;

        System.out.println("Jari-jari   : " + lingkaran.r);
        System.out.println("Luas        : " + lingkaran.area());
        System.out.println("Keliling    : " + lingkaran.circumference());
        System.out.println("1.5708 rad  : " + radiansToDegrees(1.5708) + " derajat");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModl03 ---
Jari-jari   : 7.0
Luas        : 153.93791
Keliling    : 43.98226
1.5708 rad  : 90.00028647913953 derajat
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided

```
## Petunjuk Umum
```

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: **instance field** (milik masing-masing object) dan **class field** (milik class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.

Soal — Pembersihan dan Analisis Data Suhu Mingguan

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Kebutuhan program

Class ini harus dapat:

1. **Menyimpan data suhu 7 hari**, dibuat dari array suhu mentah yang sudah ada di `main` (index array mewakili hari).
2. **Menyimpan nilai penanda data kosong (`-1.0`) sebagai aturan tetap milik class**, yaitu class field yang nilainya tidak dapat diubah dan dipakai bersama oleh semua object.
3. **Menampilkan seluruh data apa adanya**, menandai hari yang kosong secara jelas.

4. **Mencari apakah ada hari yang kosong, dan di index berapa**, karena index inilah yang nanti dipakai untuk proses pengisian data.
5. **Mengisi hari yang kosong**, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu hari sebelum dan satu hari sesudahnya.
6. **Menghitung rata-rata suhu** dari data yang sudah bersih.

Hint

> Hint di bawah ini adalah acuan minimal. Anda tetap boleh menambah method atau menyesuaikan detail selama seluruh kebutuhan di atas terpenuhi.

Nama class: `PengolahSuhu`

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
`suhuHarian`	Instance field	`double[]`	Menyimpan array suhu 7 hari; berbeda untuk setiap object
`NILAI_KOSONG`	Class field	`double`	Nilai penanda data kosong (`-1.0`); sama untuk semua object dan tidak berubah

Method yang disarankan:

Method	Kebutuhan	Return
`tampilkanData()`	3	`void`
`cariIndexKosong()`	4	`int` — index hari yang bernilai `NILAI_KOSONG`, atau `-1` apabila tidak ditemukan
`isiDataKosong()`	5	`void` — memanggil `cariIndexKosong()` di dalamnya; jika hasilnya bukan `-1`, isi index tersebut
`hitungRataRata()`	6	`double`

Rumus imputasi (pengisian data kosong) untuk time series:

Untuk index hari `i` yang kosong:

```
suhuHarian[i] = ( suhuHarian[i-1] + suhuHarian[i+1] ) / 2
```

Ini adalah metode imputasi paling dasar untuk data deret waktu, yaitu **mengisi nilai yang hilang dengan rata-rata dari titik data tetangga terdekatnya** (sebelum dan sesudah), dengan asumsi

nilai yang hilang berada di antara dua kondisi yang sudah diketahui dan tidak berubah drastis.

Data suhu (array di `main`)

Gunakan array berikut sebagai data suhu 7 hari (Hari 1 = index 0):

```java
double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };
```

Hari yang kosong: **Hari 4**.

Alur program di `main`

1. Siapkan array `suhuHarian` seperti di atas.
2. Buat object `PengolahSuhu` dari array `suhuHarian` tersebut. Tampilkan data awal (`tampilkanData()`), lalu panggil `cariIndexKosong()` dan tampilkan index yang ditemukan.
3. Jalankan `isiDataKosong()`.
4. Tampilkan data akhir yang sudah bersih (`tampilkanData()`) beserta rata-rata suhunya.
5. Tampilkan kembali isi array `suhuHarian` di `main` (bukan lewat object `PengolahSuhu`, melainkan variabel array yang sudah disiapkan di langkah 1). Amati bahwa array ini pun sudah berubah, padahal tidak pernah diubah langsung di `main`, lalu jelaskan sebabnya dalam beberapa kalimat sebagai komentar pada kode program.

Output yang diharapkan

```
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
```

```
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Yang perlu diperhatikan saat mengerjakan

- Amati baik-baik dua baris terakhir pada output di atas. Array `suhuHarian` yang dicetak itu adalah variabel di `main`, bukan diambil lewat object `PengolahSuhu` — dan nilainya sudah berubah, padahal `main` tidak pernah memanggil operasi pengubahan itu sendiri. Ini terjadi karena array bertipe **reference**: constructor `PengolahSuhu` menyimpan array yang sama persis (bukan menyalinnya), sehingga perubahan yang dilakukan `isiDataKosong()` pada object otomatis terlihat juga dari sisi `main`. Jelaskan hal ini dalam beberapa kalimat sebagai komentar pada kode program.
- Nilai penanda `-1.0` harus ditulis satu kali saja sebagai class field, lalu dipakai oleh setiap method yang membutuhkannya. Jangan mengetik ulang angka `-1.0` di dalam method.
- Rumus imputasi mengasumsikan hari sebelum dan sesudah hari yang kosong tersedia sebagai data asli (bukan ikut kosong). Data pada soal ini sudah dijamin memenuhi asumsi tersebut.

Jawaban :

```
package humaira.projectmodul03.Unguided;

import java.util.Arrays;
import java.util.Locale;

public class PengolahSuhu {
```

```
// Class field: aturan tetap milik class, dipakai bersama semua object
private static final double NILAI_KOSONG = -1.0;

// Instance field: berbeda untuk setiap object
private double[] suhuHarian;

public PengolahSuhu(double[] suhuHarian) {
    this.suhuHarian = suhuHarian; // menyimpan referensi array yang sama (tidak disalin)
}

public void tampilkanData() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            System.out.println("Hari " + (i + 1) + " : (kosong)");
        } else {
            System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "°C");
        }
    }
}

public int cariIndexKosong() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            return i;
        }
    }
    return -1;
```

```
    }

    public void isiDataKosong() {
        int i = cariIndexKosong();
        if (i != -1) {
            suhuHarian[i] = (suhuHarian[i - 1] + suhuHarian[i + 1]) / 2;
        }
    }

    public double hitungRataRata() {
        double total = 0;
        for (double suhu : suhuHarian) {
            total += suhu;
        }
        return total / suhuHarian.length;
    }

    public static void main(String[] args) {
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);
        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();
        System.out.println();
        System.out.println("Index hari kosong (dimulai dari 0): " +
pengolah.cariIndexKosong());
        System.out.println();
```

```
        pengolah.isiDataKosong();


        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.println();
        System.out.println(String.format(Locale.US, "Rata-rata : %.2f°C",
pengolah.hitungRataRata()));
        System.out.println();


        System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));
        System.out.println("(ikut berubah: constructor menyimpan referensi array yang
sama)");


        /*
         * PENJELASAN:
         * Array di Java bertipe reference. Variabel suhuHarian di main hanya
         * menyimpan alamat memori tempat array berada, bukan isi array itu sendiri.
         * Saat new PengolahSuhu(suhuHarian) dipanggil, yang diteruskan ke
         * constructor adalah alamat tersebut, lalu constructor menyimpannya apa
         * adanya ke field this.suhuHarian tanpa membuat salinan. Akibatnya,
         * variabel di main dan field di dalam object menunjuk ke satu array yang
         * sama di memori. Ketika isiDataKosong() mengubah suhuHarian[i] lewat
         * object, perubahan itu terjadi pada array yang sama, sehingga langsung
         * terlihat juga dari variabel suhuHarian di main. Agar array di main tidak
         * ikut berubah, constructor harus menyalin array, misalnya dengan
         * this.suhuHarian = suhuHarian.clone();
         */
```

```
    }

}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModl03 ---
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program ini menggunakan class `PengolahSuhu` untuk membersihkan dan menganalisis data suhu mingguan. Class tersebut memiliki dua jenis field, yaitu instance field `suhuHarian` yang bersifat `private` untuk menyimpan array suhu milik masing-masing object, dan class field `NILAI_KOSONG` yang bersifat `private static final` sebagai penanda data kosong (`-1.0`) yang sama bagi semua object dan tidak dapat diubah. Constructor menerima array suhu dan menyimpannya dengan keyword `this` karena nama parameternya sama dengan nama field. Method `tampilkanData()` menampilkan seluruh data dan menandai hari yang kosong, `cariIndexKosong()` mencari index hari yang bernilai

`NILAI_KOSONG` (mengembalikan `-1` jika tidak ada), `isiDataKosong()` memanggil `cariIndexKosong()` lalu mengisi data kosong dengan rata-rata suhu hari sebelum dan sesudahnya, dan `hitungRataRata()` menghitung rata-rata suhu dari data yang sudah bersih. Setelah `isiDataKosong()` dijalankan, array `suhuHarian` di `main` ikut berubah karena array bersifat reference, sehingga constructor menyimpan alamat array yang sama dan bukan salinannya. Dengan demikian, perubahan yang dilakukan melalui object juga terlihat dari variabel di `main`.

D. Kesimpulan

Dari Modul 3 dapat disimpulkan bahwa pemrograman berorientasi objek (PBO) membangun program dari objek-objek yang menggabungkan data dan method dalam satu kesatuan, berbeda dengan pemrograman prosedural yang berfokus pada urutan prosedur. PBO memiliki lima karakteristik utama: abstraksi untuk menentukan atribut dan perilaku yang esensial, enkapsulasi untuk melindungi data internal dan menjaga modularitas, pewarisan untuk menggunakan ulang karakteristik kelas induk, polimorfisme untuk satu antarmuka dengan banyak aksi, serta *message* untuk komunikasi antar objek. Dalam Java, kelas berfungsi sebagai cetak biru yang berisi *field* dan *method*, sedangkan objek adalah hasil instansiasi kelas yang dibuat dengan `new`. *Field* terbagi menjadi *class field* (`static`, dipakai bersama semua objek) dan *instance field* (dimiliki masing-masing objek), sementara *method* dapat berupa fungsi yang mengembalikan nilai atau prosedur (`void`).

Selain itu, praktikan memahami bahwa tipe data primitif dilewatkan ke *method* secara *pass by value*, sehingga variabel asal tidak berubah, sedangkan *array* dan objek dilewatkan dengan menyalin alamat referensinya, sehingga perubahan di dalam *method* ikut terlihat dari luar. Keyword `this` berguna untuk membedakan *field* dari parameter yang bernama sama dan untuk memanggil konstruktor lain dalam kelas yang sama. Konstruktor sendiri dipakai untuk menginisialisasi objek, dan jika diberi *modifier* `private`, objek hanya dapat dibuat dari dalam kelas itu sendiri. Pada soal *unguided*, konsep-konsep ini diterapkan lewat kelas `PengolahSuhu`, yang menunjukkan enkapsulasi dengan *field* `private`, penggunaan *class field* konstan, serta efek referensi *array* yang membuat perubahan di dalam objek juga tampak di `main`.